



Inspiring Excellence

CSE340

Summer 2025

Assignment 3

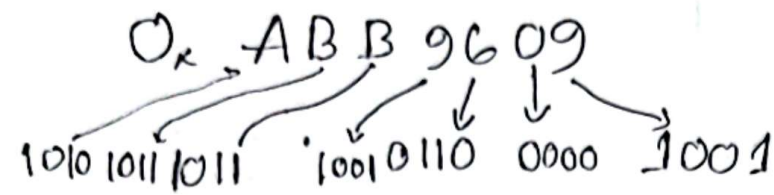
Name : Abdullah Al Fahad

ID : 21201240

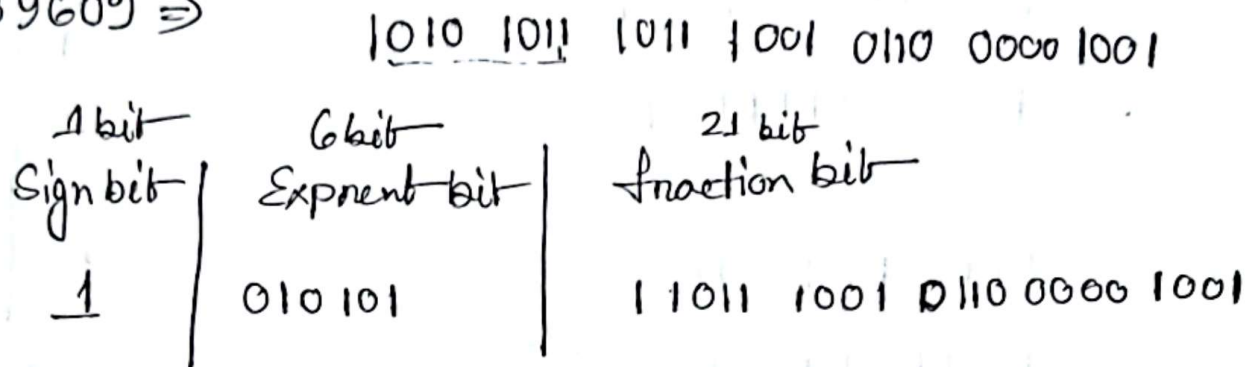
Section : 8

Faculty : PBK

Ans to the Ques no 1



0x-ABB9609 $\Rightarrow$



Biased exp $\Rightarrow$ 010101 $\Rightarrow$ 21

Bias $\Rightarrow 2^{n-1} - 1 \Rightarrow 2^{6-1} - 1 \Rightarrow 31$

Actual Exponent $\Rightarrow 21 - 31 \Rightarrow -10$

Fraction $\Rightarrow$ 0.110111001011000001001

$\Rightarrow$ 0.8620648384

Decimal $\Rightarrow (-1)^1 \times (1 + 0.8620648384) \times 2^{-10}$

$\Rightarrow -1.8620648384 \times 2^{-10}$

$\Rightarrow -1.818422694 \times 10^{-3}$

Ans

Ans to the Ques no 2

$$\alpha) 50.7869 \Rightarrow 110010.11001$$

$$\Rightarrow 1.1001011001 \times 2^5$$

$$79.83 = 1001111.11010$$

$$\Rightarrow 1.00111111010 \times 2^6$$

$$29.58 \Rightarrow 11101.10010$$

$$\Rightarrow 1.110110010 \times 2^4$$

1st Part

~~$$50.7869 - 79$$~~

$$79.83 - 29.58$$

$$\Rightarrow 1.00111111010 \times 2^6 - 1.110110010 \times 2^4$$

$$\Rightarrow 1.00111111010 \times 2^6 - 0.01110110010 \times 2^6$$

$$\Rightarrow 0.10001001000 \times 2^6$$

$$\Rightarrow 1.0001001000 \times 2^7$$

.7869	
1	.5738
1	.1476
0	.2952
0	.5904
1	.1808
0	.3616
0	.7232
1	.4464

.58		0.83	
1	.46	1	.66
0	.32	1	.32
0	.64	0	.64
1	.28	1	.28
0	.56	0	.56

2nd Part

2

$$50.7869 + (7783 - 2958)$$

$$\Rightarrow 1.1001011001 \times 2^5 + 1.0001001000 \times 2^7$$

$$\Rightarrow 0.011001011001 \times 2^7 + 1.0001001000 \times 2^7$$

$$\Rightarrow 1.01110111001 \times 2^7$$

b) $64.2486 \Rightarrow 1000000.00111$
 $\Rightarrow 1.00000000111 \times 2^6$

$49.1832 \Rightarrow 110001.00101$
 $\Rightarrow 1.1000100101 \times 2^5$

$$\Rightarrow 1.00000000111 \times 2^6 \wedge 1.1000100101 \times 2^5$$

$$\Rightarrow (1.00000000111 \times 1.1000100101) \times 2^{6+5}$$

$$\Rightarrow 1.100010101001100000011 \times 2^{11}$$

	.246
	$\times 2$
0	.492
	$\times 2$
0	.984
	$\times 2$
1	.968
	$\times 2$
1	.936
	$\times 2$
1	.872

	.1832
	$\times 2$
0	.3664
	$\times 2$
0	.7328
	$\times 2$
1	.4656
	$\times 2$
0	.9312
	$\times 2$
0	.8624

Ans to the Que no 3

$$28.4810 \Rightarrow 11100.01111$$

$$\Rightarrow 1.110001111 \times 2^4$$

$$4.0210 \Rightarrow 100.00000$$

$$\Rightarrow 1.00000000 \times 2^2$$

$$28.4810 - (-4.0210)$$

$$\Rightarrow 28.4810 + 4.0210$$

$$\Rightarrow 1.110001111 \times 2^4 + 1.00000000 \times 2^2$$

$$\Rightarrow 1.110001111 \times 2^4 + 0.0100000000 \times 2^4$$

$$\Rightarrow 10.000001111 \times 2^4$$

$$\Rightarrow 1.0000001111 \times 2^5$$

In Single Precision Exponent has 8 bit

Range of Biased Exponent

$$\Rightarrow 0 \text{ to } 2^8 - 1$$

$$\Rightarrow 0 \text{ to } 255$$

$$\Rightarrow 1 \text{ to } 254$$

	.4810
	$\times 2$
0	.962
	$\times 2$
1	.924
	$\times 2$
1	.848
	$\times 2$
1	.696
	$\times 2$
1	.392

	.0210
	$\times 2$
0	.042
	$\times 2$
0	.084
	$\times 2$
0	.168
	$\times 2$
0	.336
	$\times 2$
0	.672

$$\text{Now, Bias} \Rightarrow 2^{8-1} - 1 \Rightarrow 127$$

$$\text{Bias Exponent} \Rightarrow 127 + 5$$

$$\Rightarrow 132$$

which is in the range $\therefore$ No overflow/underflow occurred.

Ans to the Ques no 4

a) In IEEE 754 floating point representation the exponent field is stored as an unsigned integer but in reality exponent can be positive or negative to handle both positive and negative in a uniform way; IEEE 754 uses a bias.

Biased exponent $\Rightarrow$ ~~actual~~ actual exponent + bias

$$\text{Now, In single precision bias} = 2^{n-1} - 1 \Rightarrow 2^{8-1} - 1 \Rightarrow 127$$

$$\text{If actual exponent} = 5; \text{ Biased exponent} \Rightarrow 5 + 127 \\ \Rightarrow 132$$

OR,

$$\text{If actual exponent} = -5; \text{ Biased exponent} \Rightarrow -5 + 127 \\ \Rightarrow 122$$

So, both positive and negative exponents are both neatly encoded as unsigned integers without needing a separate "sign bit" for the exponent. This makes hardware simpler and comparison easier.

⑥ Optimized multiplication improved efficiency and performance in various ways. First ~~it use~~ use. In optimized multiplication it uses only two registers ~~insted~~ instead of three registers.

In Optimized multiplication ALU is 64 bit where in traditional multiplication ALU size is 128 bit so, 64 bit calculation is faster than ~~the~~ 128 bit calculation, ~~and~~ Moreover,

Traditional long multiplication uses 3 three registers, where optimized multiplication uses two registers which make optimized multiplication more ~~optimal~~ optimal or and simpler hardware design.

Ans to the Ques no 5

i) mul: Do signed x signed multiplication, stores lower 64 bits of the product.

Syntax: mul rd, rs1, rs2

Ex: addi x5, x0, 10

addi x6, x6, -3

mul x7, x5, x6 // x7 = -30

ii) mulh: Stores the upper 64 bits of the 128 bit product. Do signed x signed multiplication and It use to detect overflow. In signed 64 bit multiplication.

Syntax: mulh rd, rs1, rs2

Ex: mulh x7, x5, x6 // x7 = upper 64 bit of product.

iii) mulhu: It does unsigned x unsigned multiplication stores the upper 64 bit of the 128 bit product

Syntax: mulhu rd, rs1, rs2

x5 $\Rightarrow$ 0x1FFFFFFFFFFFFFFFFF

x6 $\Rightarrow$ 0x2

mulhu X_7, X_5, X_6

// X_7 = Upper 64 bits of
unsigned product.

10) mulhsu: multiply high signed/unsigned,
gives the upper 64 bits of the product
assuming one operand is signed and
the other unsigned.

Syntax: mulhsu rd, rs1, rs2

Ex: $X_5 = -5000$ $X_6 = 4000$

mulhsu X_7, X_5, X_6

Ans to the Ques 6

i) Comparison instruction like f.e.d stores boolean result (0/1) which is an integer value that's why it stores in integer register ~~which is X5~~. Because X5 stores integer value that's why X5 is an integer register.

ii) False,

Single precision is 32 bits and f.e.d here, (d) denotes double precision which is 64 bits so, f3 and f5 both are double precision floating point numbers.

iii) f.e.d X5, f3, f5

it checks if $f3 \leq f5$ if true then $X5 = 1$ or $X5 = 0$

Now, if $X5 = 0$; that's mean $f3 > f5$

Ans to the Ques no 7

freq. d x_5, f_1, f_2

Beq $x_0, x_5, \text{JumpNotEqual}$
~~JumpEqual~~

JumpEqual:

Beq x_0, x_0, Exit

Jump Not Equal:

Beq x_0, x_0, Exit

Exit:

Ans to the Ques 8

fld x5, f1, f2 // $f_1 \leq f_2$

Beq x0, x5, jumpGreater

Jump NotBroken;

srli x19, x19, 4 // ~~x19 = C~~
// $x_{19} = x_{19}/4$

Beq x0, x0, exit

Jump Greater!

Exit!